1. Ontology

Let's begin by defining our ontology. A basic understanding of the terms used in the PTA-ATIP framework is assumed. We leave the exact semantics of the predicate/subject division open for interpretation. Different implementations of the ATIP may divide the same sentence differently. It remains to be seen if using e.g. phrase structure grammar or dependency grammar will result in different classifications.

Premises, conclusions, and

set	description
N	The set of all names (alias argument types, symbols) as defined by the PTA.
Σ	The class of all sentences.
$\mathbb{P}; \rho = (s, p) \in \mathbb{P}$	The class of all premises.
$\mathbb{C}; c = (s, p) \in \mathrm{C}$	The class of all conclusions.
$\mathrm{Pc} \supset (\mathbb{P}, \mathbb{C})$	The class of all propositional content.
$P; p \in P = [c_i : c_i \in \mathrm{Pc}]$	The class of all complete propositions.
$\mathrm{Pr}; p \in \mathrm{Pr}$	The class of all predicates.
$\mathrm{Su}; s \in \mathrm{Su}$	The class of all subjects.
$R_{(\tau_1,...,\tau_n)}; R(t_1, ..., t_n) \in R_{(\tau_1,...,\tau_n)}$	The class of all n-ary relations, where $n \in \mathbb{N}$ and d denotes the types of each argument.
$\mathbb{F} \coloneqq \{\alpha, \beta, \gamma, \delta\}$	The set of all *forms.*
$\mathbb{S} \coloneqq \{S_\alpha, ..., S_\delta\}$	The set of all *substances.*
$\mathbb{L} \coloneqq \{L_\alpha, ..., L_\delta\}$	The set of all *levers.*
$\mathrm{Sr} \coloneqq \{M_{\mathrm{ac}}, M_{\mathrm{as}}, M_i, M_p\}$	The set of all types of 'surrounding elements,' to be stripped from the sentence during canonicalization. The subscripts of each subset refer to the 4 kinds of surrounding elements: argumentative coherence markers, argumentative structure markers, illocutive markers, and perlocutive markers. This is basically a bucket for "everything else," though these markers play a role in the final naming procedure.
$\mathbb{A} = \{A, U\}$	Acceptability relations of a premise.

2. The procedure

This consists of transformations over the various domains listed in Section 1.

2.1. Part I. Standardization

This part requires a lot of NLP translation. It is a form of transformative analysis.

Let's denote our original sentence as σ.

2.1.1. Step 1 - Parsing

Step 1 is to identify the subject-predicate form of the σ. This is done through the semantics of the chosen syntax grammar. Consider this a function

$$g(\sigma) : \Sigma \to \text{AST} \Rightarrow a$$

where "AST" refers to an abstract syntax tree. "g" is short for "grammar."[1] The AST is a tree graph $G = (V, E)$ where $e \in E \to e \in \text{Pr} \cup \text{Su} \cup \text{Pc}$.

2.1.2. Step 2 - Canonicalization

Step 2 is canonicalization, a translation of a declarative sentence of arbitrary form to the form

$$c \because \rho = (s_1, p_1) \because (s_2, p_2) \quad (\star)$$

(This can equivalently be written $\rho \therefore c$.) We define κ as our canonicalization function.

$$\kappa(a) : \text{AST} \to \text{AST} \Rightarrow a'$$

a' is our *canonical form in tree representation.* To transform a to a' we strip out any surrounding elements, resolve any enthymemes, and rearrange the sentence to match $(\star)$. A further list of possible steps in the canonicalization procedure is enumerated in ATIP §1.2.

Flattening the AST back into a string gives us the *canonicalized sentence* σ'.

2.2. Part II. Characterization

2.2.1. Step 3 - Form

We define the forms as follows.

name	symbolic repr.
α	$X.a \because Y.a$
β	$X.a \because X.b$
γ	$X.a \because Y.b$
δ	$\xi.q \because Z.q$, where $\xi \in \mathbb{A}$

where $X, Y, Z, ... \in \text{Re}$, $a, b, c, ... \in \text{Su}$, $p, q, r, ... \in \text{Pr}$.

Define the procedure f mapping a canonicalized sentence σ' to $\mathbb{F}$ using following the mapping above. Let C be the class of canonicalized sentences.

$$f(\sigma') : C \to \mathbb{F}$$

We can equivalently define this over the canonicalized AST, a'.

$$f(a') : \text{AST} \to \mathbb{F}$$

2.2.2. Step 4 - Substance

Each form gets its own substance type $\mathbb{T}_F$, as follows.

$$\mathbb{T}_\alpha = \{F, V, P\}$$
$$\mathbb{T}_\beta = \{s, p, u\}$$
$$\mathbb{T}_\gamma = \{i, d\}$$
$$\mathbb{T}_\delta = \{\text{I}, \text{II}, \text{III}\}$$

Then we may define a transformation S_F as follows,

[1] This could be inaccurate. If so then just replace this with whatever computational linguists actually do.

$$
\begin{aligned}
S_\alpha((p_a, c_a)) &: \mathrm{Pr}^2 \to \mathbb{T}_\alpha^2 \\
S_\beta((s_a, s_b)) &: \mathrm{Su}^2 \to \mathbb{T}_\beta^2 \\
S_\gamma(R_s, R_p) &: \left(R_{(\mathrm{Su},\mathrm{Su})}, R_{(\mathrm{Pr},\mathrm{Pr})}\right) \to \mathbb{T}_\gamma \\
S_\delta(q, Z) &: (P, R_P) \to \mathbb{T}_\delta
\end{aligned}
$$

where Z denotes a relationship that applies to the full proposition q.

The exact procedures for the transformations above are outlined in ATIP §2.4.

2.2.3. Step 5 - Lever

Levers require a substance type and a keyword, $k \in K_F$, as defined in ATIP §2.5. Keywords are defined in CAPITAL LETTERS and are binary relations between predicates.

Define a mapping l_F as follows:

$$
\begin{aligned}
l_\alpha((t_1, t_2), k) &: (\mathbb{T}_\alpha^2, K_\alpha) \to L_\alpha \\
l_\beta((t_1, t_2), k) &: \left(\mathbb{T}_\beta^2, K_\beta\right) \to L_\beta \\
l_\gamma(t, k) &: \left(\mathbb{T}_\gamma, K_\gamma\right) \to L_\gamma \cup \varnothing \\
l_\delta(t, k) &: (\mathbb{T}_\delta, K_\delta) \to L_\delta
\end{aligned}
$$

where L_F is defined in ATIP §2.5. Note that $l_\gamma(d, k) = \varnothing$ for all k. This is called a non-sequitur.

2.3. Part III.

Here we bring it all together and define the full pipeline.

2.3.1. Step 6 - Naming

We define the naming function n_F as follows.

$$
n_F(\sigma', s, l) : (C, \mathbb{S}, \mathbb{L}) \to N
$$

where $s = S_F$, $l = L_F$.

We first find the substance type $t \in \mathbb{T}_F$ by applying $s(\sigma')$. Then we feed this into l, yielding $\lambda \in L_F$. Finally, we consult table 7 in ATIP §2.6 to translate (F, t, λ) to the argument's symbol.